TΔR: A Browser-Native Architecture for Real-Time Air Traffic Reconstruction, Rule-Based Classification, and Client-Side Diagnostics

Technical White Paper

Cina Naqshbandi
Independent Researcher, Cinascorp Research
cinascorp@gmail.com

Repository: https://github.com/cinascorp/TAR
Deployment: https://cinascorp.github.io/TAR/

November 2026

Abstract

TΔR (The Airway Reconstruction) is an open-source web application that reconstructs and displays live air traffic entirely inside the browser. It is a single static HTML file: no application server and no database are required. The application retrieves aircraft state records from an unofficial public flight-data feed, adds contacts from the OpenSky Network states API, synchronises a military-aircraft index from the community ADS-B.lol service, normalises everything into an in-memory index keyed by the aircraft hexadecimal identifier, assigns each aircraft to one of eleven display classes (plus an optional twelfth) with a rule cascade, and draws the result on an OpenLayers map with continuous animation between feed updates.

This paper documents the architecture and algorithms as implemented in the current revision of the source: feed acquisition, merging and parsing; classification rules; geodetic helpers; an alpha-beta tracking filter; latency-compensated dead-reckoned animation; viewport-aware rendering; and constant-heading path projection. It also describes four experimental client-side diagnostics (innovation entropy, a response-text ratio, a timestamp residual, and an entropy-threshold alert) and analyses their mathematical definitions. We show what each quantity measures, and that none of them constitutes physical detection of an aircraft. A source review lists defects and limitations with proposed corrections, among them a speed-unit assumption that, if the feed reports knots, inflates speeds at or below 250 by a factor of about 1.94. We close with an evaluation protocol for latency, frame time, memory growth, and classification accuracy.

TΔR is presented as a reproducible, deployable baseline for lightweight traffic visualization and as a testbed for transport-layer timing analysis. It is not a surveillance or detection system.

Keywords: air traffic visualization; ADS-B; flight tracking; serverless web architecture; OpenLayers; alpha-beta filter; dead reckoning; Shannon entropy; timestamp analysis.

1 Introduction

Public flight-tracking services have made aircraft positions widely visible, but most deployments depend on server-side aggregation, proprietary clients, or both. Research prototypes in air traffic visualization often inherit the same dependencies, which raises the cost of reproducing, auditing, and adapting them.

TΔR explores the opposite design point. All application logic runs in the browser: the page requests records from external feeds, parses and merges them, classifies each aircraft, estimates motion between updates, and renders the result. Deployment reduces to hosting one static file. This makes the system easy to inspect and reproduce, at the cost of depending on external services over which the application has no control.

The contributions of this paper are:

1. A complete description of a single-file, serverless architecture for live air traffic visualization, including its client-side multi-source merge (Sections 3 and 4).
2. A documented rule cascade for aircraft classification, with its evidence sources and evaluation order (Section 5).
3. A description of the geodetic, motion-estimation, and rendering layers, including an analysis of how the tracking filter and the latency compensation are and are not used by the renderer (Section 6).
4. A formal analysis of four experimental diagnostics, stating precisely what each computes and what it cannot support (Section 7).
5. A source-review list of known defects with proposed corrections, and an evaluation protocol (Sections 8 and 9).

Throughout, we distinguish what the code calculates, what the interface displays, and what independent experiments could establish. Claims are limited to what the source supports. The paper describes one revision of the source file, identified by size and SHA-256 digest in Appendix B; function names are given in `monospace` to allow direct lookup.

2 Related Work and Positioning

ADS-B research infrastructure. The OpenSky Network collects Mode S and ADS-B messages from a distributed receiver network and makes them available for research [1]. Open-source decoders such as `pyModeS` and the accompanying reference text on 1090 MHz signals describe how raw messages become the position, velocity, and identification fields that downstream tools consume [2, 3]. Security analyses of ADS-B examine spoofing and injection threats to such data [4]. TΔR does not receive or decode radio signals; it consumes records already aggregated by third-party services.

Aggregated feeds. Commercial and community aggregators expose position records through web endpoints. TΔR uses an unofficial Flightradar24 feed endpoint for the primary stream, the OpenSky REST API for supplementary contacts [12], and the community ADS-B.lol API for a military-aircraft index and the LADD list. All are external dependencies whose availability, schema, rate limits, and terms of use may change.

Tracking and geodesy. The alpha-beta filter is a classical fixed-gain tracker [5]; it is a steady-state special case of the Kalman filter [6], without an explicit covariance [7]. For distances, the haversine formula treats the Earth as a sphere, whereas ellipsoidal geodesic algorithms such as Vincenty's and Karney's are more accurate [8, 9]. Geoid models such as EGM2008 describe the separation between the geoid and the reference ellipsoid [10].

Positioning. TΔR occupies a narrow niche: a client-only visualizer and classifier that can be audited in a single file, plus an experimental sandbox for studying timestamp and innovation statistics. It makes no accuracy claim relative to receiver networks.

3 System Architecture

3.1 Single-file deployment

The interface structure, CSS, JavaScript logic, and library references are contained in one HTML document. The browser executes the application and contacts external services directly. Table 1 lists the declared dependencies and their roles. These are separate services with separate roles, not one integrated aviation data provider. A "View 3D" button in the aircraft popup opens a separate page (`3d.html`) that is not part of the reviewed file.

Dependency	Role
OpenLayers 10.11.0	Map rendering and vector-layer management
Bootstrap 5.3.8	Interface components and off-canvas panels
OpenStreetMap tiles	Base-map imagery; the dark theme is a CSS filter on the same tiles
Flightradar24 feed endpoints	Primary flight-record retrieval (two hosts)
OpenSky Network states API	Supplementary contacts, requested directly from the browser
ADS-B.lol	Military aircraft index (`/v2/mil`) and LADD list (`/v2/ladd`)
OSM Nominatim	Reverse lookup of country for a coordinate
Open-Elevation	Terrain elevation, used for drone altitude above ground
Third-party CORS relays	Optional routes when direct access is blocked (`allorigins.win`; `cors.isomorphic-git.org`)

Table 1: External dependencies declared in the source.

When the proxy option is enabled, Flightradar24 requests are routed through third-party CORS relays; the military and LADD requests fall back to one relay when the direct request fails. Those relays can observe the request, including the viewport bounds; this is a privacy consideration for deployers.

3.2 Processing flow

Figure 1 shows the main data path. A refresh (`refresh`) derives the geographic bounds of the viewport (`bFV`), requests records and merges the OpenSky contacts (`fetchFR24`), parses them (`parseFR24`), updates the index, selects the viewport slice, classifies and styles contacts (`viewportFlights`, `categorize`, `styleFor`), and updates animation targets (`sFT`). A refresh runs every 3 s, on demand, and 650 ms after each map movement. A separate `requestAnimationFrame` loop (`frame`) advances displayed positions independently of network timing. The military index is synchronised on its own schedule and writes into the same index. Diagnostics (`runQuantumChecks`) run after each successful refresh.

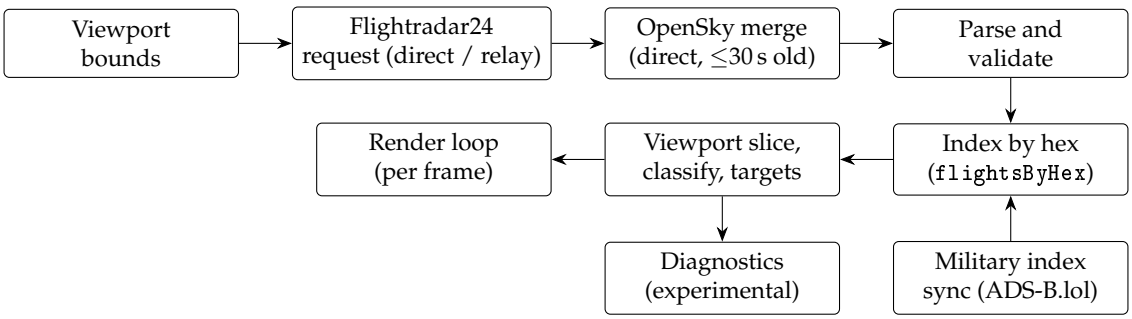


Figure 1: Principal data flow. Network refresh (every 3 s nominally) and rendering (per animation frame) are decoupled; the military index writes into the same index on a 30 s schedule.

3.3 In-memory state

Operational state lives in JavaScript collections (Table 2). The implementation does not maintain a persistent historical database; history is limited to in-memory traces.

Structure	Content
`flightsByHex`	Current aircraft records keyed by hexadecimal identifier
`sortedHexes`	Sorted identifiers used for grouping and display
`selectedFlights`	Set of user-selected identifiers
`tracksByHex`	Feed-position history appended for each aircraft in the viewport slice at every refresh (capped at 26 000 per aircraft)
`trailsByHex`	Rendered-position samples appended during animation for every animated aircraft (capped at 26 000 per aircraft); this is what the track line draws
`animF`	Per-aircraft animation state (feature, filter, position, velocity, target)
`milHexSet`, `dynMilPrefixes`, `milMeta`	Military identifiers, learned callsign prefixes, and metadata from synchronisation
`laddHexSet`	LADD identifiers, fetched when LADD mode is switched on
`countryCache`, `elevationCache`	Cached reverse-geocoding and elevation results
`jitterBuf`	Rolling buffer of filter-innovation magnitudes (at most 1000)
`lastBlobInfo`	Text, length, and reception time of the last primary-feed response

Table 2: Principal in-memory structures.

4 Data Acquisition and Parsing

4.1 Primary feed: request and fallback

`fetchFR24` builds a query from the viewport bounds and enables feed classes including ADS-B, MLAT, FLARM, satellite, ground, vehicles, estimated positions, UAV, military, gliders, and LADD, with a maximum record age parameter of 14 400 s. It tries two feed hosts in turn; for each host it attempts a direct request and, if the proxy option is on, two relayed requests. The first response that parses as a JSON object is accepted, and its text, character length, and reception time are stored in `lastBlobInfo`. If every attempt fails, the refresh is abandoned and no record is fabricated.

4.2 OpenSky merge

After a successful primary response, `fetchFR24` makes one further request, directly to the OpenSky states endpoint, with the viewport bounds as the query box. The merge is performed on the client, inside the same function, before parsing:

1. The hexadecimal identifiers already present in the primary payload are collected (lowercased).
2. Each OpenSky state is skipped if its identifier is already present, if it lacks a position, if the position is out of range, or if its last-contact time is more than 30 s old. The primary feed therefore wins whenever both sources report an aircraft.
3. Each remaining state is converted to the 19-element array layout of the primary feed: altitude from metres to feet, velocity from m/s to knots, vertical rate from m/s to ft/min, squawk from the state vector, the source label `opensky`, and the callsign duplicated into

the flight-number and callsign slots. The last-contact time (whole seconds) becomes the record timestamp.

Any error inside this block is caught and ignored, so the primary payload is returned unchanged when OpenSky is unreachable or refuses the request. By the source comments, the merge is intended to recover contacts that the primary feed withholds; the paper does not evaluate how many it recovers (Section 9). The response-text snapshot used by the diagnostics is taken before the merge and therefore describes the primary feed only. Rate limits that the provider documents for this endpoint are discussed in Section 8.

4.3 Military index and LADD synchronisation

`syncMilDB` fetches the ADS-B.lol military list at start-up (raced against a 3 s timeout in the splash sequence), every 30 s thereafter, and on demand through the MIL-SYNC button. It tries the endpoint directly and then through one relay. On a non-empty list, `ingestMil` rebuilds three structures: the set of military identifiers, a set of callsign prefixes derived from the list (two to four letters before the first digit, excluding prefixes in a configured block list of 43 airline codes), and per-aircraft metadata (type, description, operator). Any listed aircraft that carries a position is also written directly into `flightsByHex`, replacing the existing record for that identifier, so that it renders before the next primary refresh; its speed is converted with the same rule as the primary feed (Section 4.4) and its timestamp is back-dated by the reported seconds since last seen. After ingestion the index is re-sorted, the markers are redrawn, and the interface reports the contact and type counts. If synchronisation fails, the static rules continue to operate and the interface reports the offline state.

`syncLADD` runs only when LADD mode is switched on (by repeated clicks on the help-panel logo, each within 1.5 s of the previous one). It replaces the LADD identifier set from the ADS-B.lol list and writes no records. LADD classification therefore only relabels aircraft that are already present from another source; it does not add aircraft.

4.4 Record parsing

`parseFR24` iterates over array-valued entries with at least 17 elements, rejects non-numeric or out-of-range coordinates, and maps array positions to fields (Table 3). Vertical-rate elements (indices 14 and 15) are not read.

Index	Internal field	Note
0	`hex`	Identifier (or record key), lower-cased
1, 2	`lat`, `lon`	Validated to $[-90, 90]$ and $[-180, 180]$
3	`heading`	Degrees
4	`alt_ft`, `alt_m`	Feet; converted to metres
5	`spd_kmh`	Converted by `toKmH` (see below)
6, 7	`squawk`, `radar`	Transponder code; source identifier
8, 9	`acType`, `registration`	Type code; registration
10	`ts`	Source timestamp, multiplied by 1000
11, 12	`origin`, `dest`	Route endpoints
13, 16	`flightNumber`, `callsign`	Flight number falls back to index 16

Table 3: Feed array fields used by the parser.

Speed conversion. `toKmH` applies a threshold rule: a value $s \leq 250$ is multiplied by 3.6 (the factor for metres per second), and a larger value is multiplied by 1.852 (the factor for knots). The rule therefore assumes that the unit of the feed field varies with magnitude. Flightradar24

does not publish a specification for this endpoint, but community client code annotates the speed element as knots. The OpenSky merge converts velocity to knots before parsing, and the ADS-B.lol ground speed is reported in knots, and both pass through the same rule. If the field is uniformly in knots, every speed of at most 250 is overestimated by a factor $3.6/1.852 \approx 1.94$. This affects dead reckoning, the projection line, and the speed thresholds used in classification, and is listed in Section 8; Section 9 defines a direct check.

Timestamps. The source timestamp is stored in milliseconds. The local reception time is captured separately with `Date.now()` when the response body has been read, and stored with the response text in `lastBlobInfo`. Section 7 shows how the difference between these two clocks feeds the timing residual.

4.5 State update and eviction

Parsed records overwrite entries in `flightsByHex`. An aircraft absent from the latest response is removed only if it lies outside the viewport expanded by 0.8 of its extent plus 3° in latitude and 6° in longitude; absent aircraft inside that region are retained. No filter on record age was found in the reviewed code (Section 8).

4.6 Viewport-aware marker selection

Not every record in the index becomes a map feature. `viewportFlights` returns the sorted identifiers that pass the category and search filters and lie inside the current view extent expanded by 0.6 of its span plus 2° in latitude and 4° in longitude. The result is divided into groups of 26 000 identifiers, one of which is active. For the active group, `uMAT` creates or updates an animation entry per aircraft (`sFT`), appends a point to `tracksByHex`, and removes the entry and its map feature for every aircraft that is no longer in the group. On `moveend`, the cached frame extent is refreshed, the markers are restyled and re-selected from records already held in memory, and a network refresh is scheduled 650 ms later. The frame loop additionally skips map updates for entries outside a cached margin box (0.5 of the extent plus 2° and 4°), while still advancing their state and recording trail samples.

5 Aircraft Classification

5.1 Rule cascade

Classification (`categorize`) is an ordered cascade; the first matching rule decides (Table 4). The order matters: altitude overrides military evidence, and military evidence overrides the civil kinematic heuristics.

5.2 Military evidence

`isMilitary` accepts an aircraft if any of the following holds: its hex is in the synchronised military set; its hex is in a static block of 32 identifiers (`ae01a0`–`ae01bf`) that the source attributes to a USAF F-35A batch; its registration matches one of five static patterns; its callsign starts with one of 93 static prefixes or matches a prefix derived dynamically from the synchronised list (Section 4.3); or its type code is in the fighter (76 entries), transport (34), or bomber (7) sets. `milSubclass` then assigns the subclass from type tokens, also trying each token with its trailing letters stripped, and failing that from description and operator text in the synchronised metadata. Military UAV types (39 entries) are routed to the `drone` class and military helicopter types (59 entries) to the generic `military` class before subclassing.

#	Condition	Detail	Class
1	LADD mode on and hex in LADD set	Mode toggled by repeated clicks on the help-panel logo	`ladd`
2	Altitude $\geq 30\,000$ m	Uses the converted feed altitude	`highalt`
3a	Military and military UAV type		`drone`
3b	Military and military helicopter type, or helicopter heuristic		`military`
3c	Military, otherwise	Subclass from type tokens, then description text	`bomber`, `fighter`, `transport`, `military`
4	Glider type pattern, or registration pattern	Type code takes precedence when present	`glider`
5	Helicopter heuristic	Callsign pattern, $0 <$ speed < 260 km/h, $0 <$ altitude < 3000 m	`helicopter`
6	Drone heuristic	$0 <$ altitude ≤ 1200 m and $0 <$ speed ≤ 210 km/h	`drone`
7	Airline-style callsign, not a cargo prefix	Pattern of 2–3 letters, 2–4 digits, optional letter	`passenger`
8	Private registration or callsign pattern	Country-specific registration formats	`private`
9	Otherwise		`commercial`

Table 4: Classification cascade in evaluation order.

5.3 Limitations of the heuristics

The classifier is rule-based and unvalidated against ground truth. Several rules are permissive by construction: the helicopter callsign pattern is not anchored, so any callsign containing the letter H satisfies it; the drone rule is purely kinematic, uses barometric altitude above mean sea level, and low-and-slow civil aircraft can satisfy it; and static military prefixes are matched with `startsWith`, so short prefixes (including the two-letter tokens `B2`, `F3`, `LS`, and `ZZ`) and prefixes learned from the synchronised list can match unrelated callsigns. The squawk code is parsed and stored, but no rule reads it, so emergency codes do not influence classification. Accuracy, precision, and recall are therefore unknown until measured (Section 9). LADD classification only labels aircraft already present in the index; it does not detect aircraft absent from it.

6 Geodesy, Motion Estimation, and Rendering

6.1 Radius, distance, and destination

With WGS84 semi-major axis a and first eccentricity squared e^2, `wgs84Radius` computes the meridional radius of curvature

$$M(\varphi) = \frac{a\,(1 - e^2)}{\left(1 - e^2 \sin^2 \varphi\right)^{3/2}}, \tag{1}$$

which varies from about 6335 km at the equator to 6400 km at the poles, i.e. roughly -0.56% to $+0.45\%$ around the mean Earth radius. `earthRadiusAt` adds a term `egm2008Undulation`, an ad hoc trigonometric expression bounded by about 48 m in magnitude; it is not a lookup in the EGM2008 grid, and no accuracy against that model is claimed here. Its relative effect on the radius is below 10^{-5}, which is negligible beside the choice of M and the spherical approximation. The result is a WGS84-based effective radius R.

`haversine` computes the great-circle distance with R evaluated at the midpoint:

$$d = 2R\ \mathrm{atan2}\Big(\sqrt{h}, \sqrt{1-h}\Big), \quad h = \sin^2\tfrac{\Delta\varphi}{2} + \cos\varphi_1 \cos\varphi_2 \sin^2\tfrac{\Delta\lambda}{2}. \tag{2}$$

This is a spherical approximation, not an ellipsoidal geodesic. `destPoint` uses the standard spherical direct formula to place a point at bearing θ and distance d.

6.2 Velocity from heading and speed

`headingSpeedToVel` converts heading θ and ground speed v into angular rates

$$\dot\varphi = \frac{v\cos\theta}{111000}, \qquad \dot\lambda = \frac{v\sin\theta}{111000\ \max(0.01, \cos\varphi)}, \tag{3}$$

in degrees per second with v in m/s.

6.3 Alpha-beta filter

Each animated aircraft owns an instance of `ABF` with state $(\varphi, \lambda, \dot\varphi, \dot\lambda)$ and gains $\alpha = 0.35$, $\beta = 0.0833$. For comparison, the Benedict–Bordner relation $\beta = \alpha^2/(2-\alpha)$ [5] gives 0.074 at this α. When a record with a new feed timestamp arrives, with elapsed local time $\Delta t = \max(0.5,\ t_{\text{now}} - t_{\text{prev}})$ seconds, the filter predicts, forms the innovation against the raw reported position, and corrects:

$$\hat\varphi = \varphi + \dot\varphi\,\Delta t, \qquad r_\varphi = \varphi_{\text{obs}} - \hat\varphi, \tag{4}$$

$$\varphi \leftarrow \hat\varphi + \alpha r_\varphi, \qquad \dot\varphi \leftarrow \dot\varphi + \beta r_\varphi/\Delta t, \tag{5}$$

and likewise for λ. When heading and speed are available, the velocity is then blended as $\dot{\mathbf{p}} \leftarrow 0.75\,\dot{\mathbf{p}} + 0.25\,\dot{\mathbf{p}}_{\text{obs}}$. The function returns the innovation magnitude $\sqrt{r_\varphi^2 + r_\lambda^2}$ in degrees. Three properties are worth stating. First, Δt is taken from the local clock at call time, not from the feed timestamp, so it reflects the polling interval. Second, the longitude innovation is not scaled by $\cos\varphi$, so the later conversion to metres overstates east–west error at high latitude. Third, the class also defines a `predict` method, which is never called. The filter is an alpha-beta tracker, not a probabilistic Kalman filter.

6.4 Latency compensation and animation loop

When a record is synchronised to its animation entry (`sFT`), the source computes the age of the record, $a = \min\big(10, \max(0, (t_{\text{now}} - t_{\text{rec}})/1000)\big)$ seconds, and a latency-compensated position

$$\mathbf{p}^* = \mathbf{p}_{\text{obs}} + \mathbf{v}_{\text{obs}}\,a, \tag{6}$$

where $\mathbf{v}_{\text{obs}}$ is the velocity derived from the record's heading and speed. For a new aircraft, the displayed position is initialised to $\mathbf{p}^*$ and the displayed velocity to $\mathbf{v}_{\text{obs}}$. For an existing aircraft, $\mathbf{p}^*$ is stored in the entry but only the target velocity $\mathbf{v}^* = \mathbf{v}_{\text{obs}}$ and the target heading are used afterwards.

`frame` runs under `requestAnimationFrame` with a per-frame step $\Delta t_f \le 0.1\,\text{s}$. For each aircraft it advances the displayed position with the current velocity estimate, relaxes that velocity toward the target velocity, and interpolates heading with a modular angular interpolation:

$$\mathbf{p} \leftarrow \mathbf{p} + \mathbf{v}\,\Delta t_f, \qquad \mathbf{v} \leftarrow \mathbf{v} + (\mathbf{v}^* - \mathbf{v})\min(1, 3\Delta t_f), \qquad \theta \leftarrow \theta + \Delta\theta\ \min(1, 2\Delta t_f), \tag{7}$$

with $\Delta\theta = ((\theta^* - \theta + 540) \bmod 360) - 180$. Rendered-position samples are appended to `trailsByHex` at most every 500 ms, capped at 26 000 points per aircraft.

Role of the filter and of the compensation in rendering. In the reviewed source the filter output is not used to correct the displayed position: `ABF` is updated on each new feed timestamp and its innovation is recorded for diagnostics, while the displayed position is initialised once and thereafter advanced only by the velocity relaxation above. Feed positions update the target velocity and heading but do not re-anchor the displayed position, and the latency compensation $\mathbf{p}^*$ affects the display only when an entry is created. The animation is therefore open-loop dead reckoning between entity creations, and position error can accumulate with heading, speed, or manoeuvre error (Section 8). An entry is recreated when the aircraft leaves, and later re-enters, the viewport slice.

6.5 Selection, track, and path projection

Selecting an aircraft adds a track line and a forward projection to a dedicated layer. The track (`dET`) is a dashed line through the rendered-position samples in `trailsByHex`, ending at the current displayed position. `pF` projects a constant-heading, constant-speed path from the displayed position, using the displayed heading and the feed speed, with 48 samples over a horizon that depends on map zoom (Table 5), calling `destPoint` at each step. It does not model turns, flight plans, or instructions. The projection is drawn as a thin white line under a thin cyan line.

Map zoom	Projection horizon
≥ 8	750 s
$[5, 8)$	2250 s
< 5	6000 s

Table 5: Projection horizon by zoom level.

6.6 Rendering and symbology

Classes use distinct SVG symbols. The icon scale is the product of a class factor (1.0 by default, between 1.1 and 1.25 for the special classes) and a zoom factor $\text{clamp}\big(0.45 + 0.13(z - 4),\ 0.25,\ 2.5\big)$, so icons shrink on zoom-out and grow on zoom-in. The scale and rotation are updated in place on the existing icon and the scale only when its value changes. Selected military subclasses pulse: fighters with a 900 ms period and a minimum opacity of 0.5, bombers with 1800 ms and 0.65. Altitude colour uses 1500 m steps through gray, orange, yellow, green, blue, indigo, purple, and fuchsia. Altitudes at or above 30 000 m use the high-altitude class and its own colours; this is a property of the feed value, not an independent altitude measurement. The interface is bilingual (English and Persian, with right-to-left panels) and offers a light and a dark basemap.

7 Experimental Client-Side Diagnostics

The source contains four diagnostics, invoked from a function named `runQuantumChecks`. The name is a project label. The code implements no qubits, gates, circuits, or quantum hardware; every operation below is a conventional JavaScript computation. The diagnostics receive the records of the current refresh (primary feed and OpenSky merge); records injected by the military synchronisation are not among them.

7.1 Innovation statistics and entropy

`recordJitter` converts each innovation (degrees) to an approximate length $x = 111000\,\|r\|$ in metres and appends it to a single buffer of at most 1000 samples that pools the innovations of all

aircraft. With at least 10 samples, `computeJitterStats` sorts them, builds a 20-bin histogram with bin index $\min\big(19, \lfloor 20\,x/x_{\max} \rfloor\big)$, and computes the Shannon entropy [11]

$$H = -\sum_i p_i \log_2 p_i, \qquad p_i = c_i/n, \tag{8}$$

together with the mean, median, 95th percentile, and maximum. By construction $H \le \log_2 20 \approx 4.32$ bits. H summarises the distribution of innovation magnitudes under this particular binning; because the bin width is $x_{\max}/20$, a single large outlier changes H without any change in the bulk of the distribution. It does not measure electromagnetic energy, visibility, or target presence.

7.2 The `SNR_blob` ratio

Let B_i be the character codes of the last primary-feed response text (N characters) and σ^2 the population variance of the innovation buffer. The code computes

$$\mathrm{SNR}_{\mathrm{blob}} = \frac{\sum_{i=1}^{N} B_i^2}{\sigma^2}. \tag{9}$$

The numerator grows with response length and character content, so it tracks how much text was returned (for example, how many aircraft are in view), not received signal power. The denominator has units of metres squared, so the ratio is not dimensionless. It is a software-defined quantity and not a receiver signal-to-noise ratio. The field `bytes` is the JavaScript string length, which counts characters and need not equal the transferred byte count.

7.3 Timestamp residual ("ghost" candidates)

For each record, `detectGhostTargets` computes

$$\Delta\tau = t_{Rx} - t_{Tx} - \frac{d}{c}, \tag{10}$$

where t_{Rx} is the local reception time, t_{Tx} is the record timestamp, and d is the haversine distance from the map centre to the aircraft. A record is listed when $|\Delta\tau| < 1$ ms and the response length exceeds 1000 characters. Three observations limit interpretation:

1. The map centre is a viewport parameter, not a receiver location, and the feed is aggregated by a provider from many receivers, so d/c is not the propagation delay of any actual path. It is at most about 67 ms for antipodal points and under 1 ms within 300 km.
2. $t_{Rx} - t_{Tx}$ is dominated by record age, provider processing, network transit, and client clock offset. None is calibrated in the source.
3. If the feed timestamp has one-second resolution, t_{Tx} is an integer multiple of 1000 ms, so a 1 ms window around it is rarely satisfied under normal operation; hits would be expected to reflect clock offset or timing coincidence. Timestamps of merged OpenSky records also have one-second resolution.

A listed record is a software-defined candidate. It is not evidence of an aircraft that conventional tracking misses.

7.4 Entropy-threshold alert

`ShadowDetector` keeps a history of 100 values, initialised to zero. Each refresh, `runQuantumChecks` appends $x_t = \min(1, \bar{x}/5000)$, where $\bar{x}$ is the current mean innovation in metres. Values are bucketed as $b = \lfloor 10 x_t \rfloor$, Shannon entropy H_t is computed over the bucket frequencies, and an alert is raised when

$$|H_t - 3.5| > 0.85. \tag{11}$$

Attainable range. Since $x_t \in [0, 1]$, the bucket index takes at most 11 values, so $H_t \leq \log_2 11 \approx 3.459 < 3.5$; for 100 samples the maximum is about 3.459. The baseline lies above the maximum attainable entropy, and the alert condition reduces to $H_t < 2.65$ bits, which requires roughly $2^{2.65} \approx 6.3$ equally populated buckets, i.e. mean-innovation values spread over about 3 km. Because the history starts as 100 zeros, $H_t = 0$ until real values displace the padding (about 100 refreshes, or five minutes at the nominal interval). Moreover, $\bar{x}$ is itself the mean of a rolling buffer of up to 1000 samples, so consecutive values are strongly correlated and change slowly. If typical mean innovation is small relative to 5000 m, the history occupies few adjacent buckets and the alert is expected to remain asserted. We state this as an analytical consequence of the definitions; the typical innovation level should be confirmed with the benchmark logger. In either case, the alert reflects low diversity of mean-innovation levels under a fixed baseline, and does not establish an electromagnetic shadow.

8 Known Defects and Limitations from Source Review

Table 6 lists issues identified by reading the source, with proposed corrections. They do not negate the implemented functions; they bound what can be claimed. Rows are ordered by their expected effect on what the user sees.

Table 6: Defects and limitations identified in the source.

Issue	Effect	Proposed correction
Speed rule (`toKmH`) treats values ≤ 250 as m/s; OpenSky and ADS-B.lol speeds, in knots, pass through the same rule	If the feed is in knots, speeds ≤ 250 are overestimated by $\approx 1.94\times$; dead-reckoned icons of slower aircraft run ahead of the true position, and the projection line, helicopter and drone thresholds are affected	Convert each source with its own unit (knots $\times 1.852$, m/s $\times 3.6$); verify with the speed check of Section 9
Displayed position not re-anchored to the feed or the filter; latency compensation used only at entry creation	Open-loop drift between entry creations; the error grows with time since creation	Pull the displayed position toward the compensated feed position with a bounded gain at each new record timestamp
OpenSky merge is requested every 3 s without credentials; failures are swallowed	OpenSky documents 400 credits per day for anonymous use, 1 to 4 credits per request depending on box area, and a 10 s time resolution [12]; at 3 s polling the allowance lasts roughly 5 to 20 minutes, after which the merge contributes nothing and the interface does not say so	Authenticate or relay; poll no faster than the data resolution; show merge status and yield in the interface
`ingestMil` overwrites `flightsByHex` records with ADS-B.lol records, and the next refresh overwrites them back	The same aircraft alternates between sources with different latencies; injected records are snapshots between 30 s syncs; innovation statistics mix source offsets	Keep per-source records and select by newest timestamp

Table 6 (continued)

Issue	Effect	Proposed correction
`trailsByHex` gets a sample every 500 ms for every animated aircraft, `tracksByHex` one per refresh for every aircraft in the slice, and neither is evicted	Memory grows with session length and traffic density; the 26 000-point cap bounds each aircraft, not the total	Record only for selected aircraft; evict when an aircraft is removed
Absent aircraft retained inside the expanded viewport; no age filter	Stale contacts can persist and be animated	Evict by record age and last-seen time
Filter Δt from local clock; longitude innovation not cosine-scaled	Innovation magnitude mixes polling jitter with geometry; east–west error overstated at high latitude	Use feed timestamps; scale longitude by $\cos\varphi$
Unanchored helicopter pattern; permissive military prefixes (93 static plus learned); drone rule on barometric altitude	False positives in classification; thresholds act on the speeds of the first row	Anchor patterns; validate against labelled data; use altitude above ground where available
Squawk parsed but unused	Emergency codes (7500, 7600, 7700) cannot influence classification or alerts	Add an explicit rule if emergency handling is intended
Shadow baseline 3.5 bits exceeds maximum 3.459 bits; zero-initialised history; input is a rolling mean	Alert expected to be asserted continuously	Calibrate on null data; start only after the history is full
`SNR_blob` mixes text energy with variance in m^2, on the primary-feed text only	Not a signal-to-noise ratio	Rename or remove from physical claims
Timestamp residual uses map centre and one-second timestamps	Hits not attributable to propagation	Calibrate clocks; use a known receiver location
Two class-scale tables (`baseScaleFor` in the frame loop, `baseScaleForCat` in `restyleAll`) differ for special classes	Icon size of those classes can change after a map movement	Use one table
Undulation term is not a geoid model	Name overstates the model; effect negligible	Rename or use a real geoid grid
Unofficial feed and third-party relays	Availability, terms, and privacy risk	Document terms; support an alternative source

9 Evaluation Protocol

No benchmark results are reported in this version: the reviewed source contains no instrumentation, and the figures that would support throughput or accuracy claims have not been measured. We define the protocol so that results can be reproduced from the open-source application with a lightweight logger added to the refresh and frame paths (Table 7).

Metric	Definition	Method
Fetch latency	Request start to body read, per attempt and per route	`performance.now()` around `fetchFR24`; record route and outcome
Merge yield and status	Contacts added by the OpenSky merge per refresh; share of requests answered with each HTTP status, including 429	Counters inside the merge block
Parse and update time	Time in `parseFR24`, index update, and `uMAT`	`performance.now()` deltas per refresh
Frame time	Interval between animation frames; report median and 95th percentile	Timestamps in `frame`
Contacts rendered	Records per refresh and features on the map	Counts per refresh at several zoom levels
Memory growth	Total points held in `trailsByHex` and `tracksByHex`, and heap size, over a long run	Periodic counters; browser memory tools
Route reliability	Share of refreshes served by each direct or relayed route	Success and failure counters
Speed-unit check	Reported speed versus speed derived from successive positions of the same aircraft, split at 250	Compare `spd_kmh` with the distance between consecutive record positions over the elapsed record time
Classification accuracy	Precision and recall per class against labelled reference data	Offline comparison on logged snapshots
Position error	Displayed versus next observed position at each refresh	Distance between `ent.pos` and the next feed position
Innovation baseline	Distribution of innovation and of H_t in normal operation	Logger output over long runs

Table 7: Proposed metrics.

Runs should record device, browser and version, network type, zoom level, viewport size, duration, and the SHA-256 digest of the source file (or the commit hash), and be repeated across at least a desktop and a mobile device. Reference data for classification should come from a source independent of the rules being tested.

10 Conclusion and Development Direction

TΔR shows that a complete live-traffic visualizer, with feed acquisition and merging, classification, geodesy, motion estimation, and diagnostics, can run as one static file in the browser. Its value is as a transparent, reproducible baseline. Its experimental diagnostics are executable and measurable, but as defined they do not support physical detection claims, and the shadow alert in particular requires recalibration.

The near-term plan follows from Section 8: settle the speed unit with the check of Section 9 and use one conversion per source, close the loop between the feed and the displayed position, bound the trail histories, handle the OpenSky quota explicitly, add age-based eviction, recalibrate or retire the entropy alert, and publish the benchmark results. Longer-term work includes comparison against independent receiver data, per-class accuracy against labelled references, and an alternative data source. Any future claim about timing-based detection would require synchronised clocks, a characterised data path, and controlled observations.

Acknowledgements

The author thanks the OpenSky Network, the ADS-B.lol community, and the OpenStreetMap community for the open data and infrastructure that the application builds on.

Code and Data Availability

The source is available at https://github.com/cinascorp/TAR and the live deployment at https://cinascorp.github.io/TAR/. The application ingests live third-party feeds; no dataset is bundled and no measurement dataset is reported in this paper.

References

[1] M. Schäfer, M. Strohmeier, V. Lenders, I. Martinovic, M. Wilhelm. Bringing up OpenSky: A large-scale ADS-B sensor network for research. In *Proc. 13th Int. Symp. on Information Processing in Sensor Networks (IPSN)*, 2014.

[2] J. Sun, H. Vû, J. Ellerbroek, J. M. Hoekstra. pyModeS: Decoding Mode-S surveillance data for open air transportation research. *IEEE Trans. Intelligent Transportation Systems*, 21(7), 2020.

[3] J. Sun. *The 1090 Megahertz Riddle: A Guide to Decoding Mode S and ADS-B Signals*. TU Delft OPEN Publishing, 2021.

[4] M. Strohmeier, V. Lenders, I. Martinovic. On the security of the Automatic Dependent Surveillance-Broadcast protocol. *IEEE Communications Surveys & Tutorials*, 17(2), 2015.

[5] T. R. Benedict, G. W. Bordner. Synthesis of an optimal set of radar track-while-scan smoothing equations. *IRE Trans. Automatic Control*, AC-7(4), 1962.

[6] R. E. Kalman. A new approach to linear filtering and prediction problems. *J. Basic Engineering*, 82(1), 1960.

[7] Y. Bar-Shalom, X. R. Li, T. Kirubarajan. *Estimation with Applications to Tracking and Navigation*. Wiley, 2001.

[8] T. Vincenty. Direct and inverse solutions of geodesics on the ellipsoid with application of nested equations. *Survey Review*, 23(176), 1975.

[9] C. F. F. Karney. Algorithms for geodesics. *J. Geodesy*, 87(1), 2013.

[10] N. K. Pavlis, S. A. Holmes, S. C. Kenyon, J. K. Factor. The development and evaluation of the Earth Gravitational Model 2008 (EGM2008). *J. Geophysical Research*, 117(B4), 2012.

[11] C. E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3), 1948.

[12] The OpenSky Network. REST API documentation (state vectors, limitations for anonymous users, API credit usage). https://openskynetwork.github.io/opensky-api/rest.html, accessed November 2026.

A Implementation Constants

Quantity	Value	Context
Feed refresh interval; move debounce	3000 ms; 650 ms	`startAutoUpdate`; `moveend`
Military synchronisation interval	30 000 ms (also at start-up, on demand)	`syncMilDB`
Maximum record age requested	14 400 s	Feed query
OpenSky age limit; compensation clamp	30 s; 10 s	`fetchFR24`; `sFT`
Alpha, beta; filter Δt floor	0.35, 0.0833; 0.5 s	`ABF`
Velocity blend (previous, observed)	0.75, 0.25	`ABF`
Frame step cap; velocity and heading rates	0.1 s; 3 s^{-1}, 2 s^{-1}	`frame`
Trail interval; maximum points	500 ms; 26 000	`frame`, `aTT`
Group size	26 000	`uMAT`
Slice margin; frame margin; eviction margin	0.6, 0.5, 0.8 of extent	`viewportFlights`, `refreshFrameExtent`, `refresh`
Fixed margins (lat, lon) for the three above	2°, 4°; 2°, 4°; 3°, 6°	same
Projection samples	48	`pF`
High-altitude threshold	30 000 m	`isHighAlt`
Altitude colour step; colours	1500 m; 8	Symbology
Zoom scale	$\mathrm{clamp}(0.45 + 0.13(z - 4), 0.25, 2.5)$	`zoomScale`
Innovation buffer size	1000	`jitterBuf`
Histogram bins; minimum samples	20; 10	`computeJitterStats`
Shadow history; baseline; threshold	100; 3.5; 0.85	`ShadowDetector`
Shadow input normalisation	5000 m	`runQuantumChecks`
Timestamp-residual window; minimum length	1 ms; 1000 characters	`detectGhostTargets`
Static sets: prefixes; airline block	93; 43	`MIL_PREFIXES`, `AIRLINE_BLOCK`
Static sets: fighter; transport; bomber	76; 34; 7	type sets
Static sets: military helicopter; military UAV	59; 39	type sets
Static F-35A hex block; cargo prefixes	32; 6	`F35_HEXES`, `CARGO_PREFIX`

Table 8: Constants as implemented.

B Reviewed Artefact

The paper describes the file `tar.html` with 4227 lines and 174 108 bytes. Its SHA-256 digest is

```
52fb188f67df609955dc7d42d88787a8c948d0a3cc133f8ebb6169b9e8c248cb
```